

Multimodal Deep Learning

Solutions to Exercise Sheet 4

Date: 27th of May, 2025

Exercise 1: Groups

For each set S and binary operation $*$ below, determine whether $(S, *)$ forms a group. If yes, verify all four group axioms. If no, identify which axiom fails with a counterexample.

(a) $(\mathbb{N}_0, +)$, where $\mathbb{N}_0 = \{0, 1, 2, \dots\}$.

(b) $(R_S, \circ)$, with

$$R_S = \{R_0, R_{90}, R_{180}, R_{270}\}, \quad R_\theta = \text{rotation of the square by } \theta^\circ \ (0 \leq \theta < 360).$$

(c) $(\{R_\theta \mid \theta \in \mathbb{R}\}, \times)$, where

$$R_\theta = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix},$$

and the operation is matrix multiplication.

Solution

(a) $(\mathbb{N}_0, +)$, where $\mathbb{N}_0 = \{0, 1, 2, \dots\}$. We check the group axioms for $(\mathbb{N}_0, +)$. **Inverse Element:** For an element $a \in \mathbb{N}_0$, its additive inverse must be an element $a' \in \mathbb{N}_0$ such that $a + a' = 0$. This implies $a' = -a$. If $a \in \mathbb{N}_0$ and $a > 0$, then $-a$ is a negative integer. Therefore, $-a \notin \mathbb{N}_0$. For example, $1 \in \mathbb{N}_0$, but its inverse $-1 \notin \mathbb{N}_0$.

Since the inverse axiom fails, $(\mathbb{N}_0, +)$ is not a group.

(b) $(R_S, \circ)$, with $R_S = \{R_0, R_{90}, R_{180}, R_{270}\}$, where R_θ is the rotation of a square by θ° ($0 \leq \theta < 360$). The operation $\circ$ is composition of rotations. We can write $R_\alpha \circ R_\beta = R_{(\alpha+\beta) \pmod{360}}$. We check the group axioms for $(R_S, \circ)$.

1. **Closure:** Let $R_\alpha, R_\beta \in R_S$. Then $\alpha, \beta \in \{0, 90, 180, 270\}$. These are integer multiples of 90. The sum $\alpha + \beta$ is also an integer multiple of 90. Therefore, $(\alpha + \beta) \pmod{360}$ will be an angle in $\{0, 90, 180, 270\}$.

$\circ$	R_0	R_{90}	R_{180}	R_{270}
R_0	R_0	R_{90}	R_{180}	R_{270}
R_{90}	R_{90}	R_{180}	R_{270}	R_0
R_{180}	R_{180}	R_{270}	R_0	R_{90}
R_{270}	R_{270}	R_0	R_{90}	R_{180}

2. **Associativity:** Using addition modulo 360: $(R_\alpha \circ R_\beta) \circ R_\gamma = R_{(\alpha+\beta) \pmod{360}} \circ R_\gamma = R_{((\alpha+\beta)+\gamma) \pmod{360}}$. And $R_\alpha \circ (R_\beta \circ R_\gamma) = R_\alpha \circ R_{(\beta+\gamma) \pmod{360}} = R_{(\alpha+(\beta+\gamma)) \pmod{360}}$. Since addition of numbers is associative, the results are equal.

3. **Identity Element:** The element $R_0 \in R_S$ (rotation by 0°) is the identity element. For any $R_\theta \in R_S$, $R_\theta \circ R_0 = R_{(\theta+0) \pmod{360}} = R_\theta$, and $R_0 \circ R_\theta = R_{(0+\theta) \pmod{360}} = R_\theta$.

4. **Inverse Element:** For each $R_\theta \in R_S$, we seek an $(R_\theta)^{-1} = R_\phi \in R_S$ such that $R_\theta \circ R_\phi = R_0$. This requires $(\theta + \phi) \pmod{360} = 0$, which implies $\phi = (360 - \theta) \pmod{360}$.

- For R_0 : inverse is $R_{(360-0) \pmod{360}} = R_0 \in R_S$.
- For R_{90} : inverse is $R_{(360-90) \pmod{360}} = R_{270} \in R_S$.
- For R_{180} : inverse is $R_{(360-180) \pmod{360}} = R_{180} \in R_S$.
- For R_{270} : inverse is $R_{(360-270) \pmod{360}} = R_{90} \in R_S$.

Each element in R_S has an inverse in R_S . The inverse axiom holds.

Since all four group axioms are satisfied, $(R_S, \circ)$ is a group. It is isomorphic to the cyclic group $\mathbb{Z}_4$.

(c) $(\{R_\theta \mid \theta \in \mathbb{R}\}, \times)$, where $R_\theta = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$, and the operation is matrix multiplication. Let $S = \{R_\theta \mid \theta \in \mathbb{R}\}$. We check the group axioms for $(S, \times)$.

1. **Closure:** For any $R_\alpha, R_\beta \in S$, their product is

$$\begin{aligned} R_\alpha R_\beta &= \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix} \begin{pmatrix} \cos \beta & -\sin \beta \\ \sin \beta & \cos \beta \end{pmatrix} \\ &= \begin{pmatrix} \cos \alpha \cos \beta - \sin \alpha \sin \beta & -(\cos \alpha \sin \beta + \sin \alpha \cos \beta) \\ \sin \alpha \cos \beta + \cos \alpha \sin \beta & -\sin \alpha \sin \beta + \cos \alpha \cos \beta \end{pmatrix} \\ &= \begin{pmatrix} \cos(\alpha + \beta) & -\sin(\alpha + \beta) \\ \sin(\alpha + \beta) & \cos(\alpha + \beta) \end{pmatrix} = R_{\alpha+\beta}. \end{aligned}$$

Since $\alpha, \beta \in \mathbb{R}$, their sum $\alpha + \beta \in \mathbb{R}$. Thus, $R_{\alpha+\beta} \in S$. Closure holds.

2. **Associativity:** Matrix multiplication is an associative operation. Therefore, $(R_\alpha R_\beta) R_\gamma = R_\alpha (R_\beta R_\gamma)$ for all $R_\alpha, R_\beta, R_\gamma \in S$. Associativity holds.

3. **Identity Element:** Consider $R_0 = \begin{pmatrix} \cos 0 & -\sin 0 \\ \sin 0 & \cos 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I_2$. Since $0 \in \mathbb{R}$, $R_0 \in S$. For any $R_\theta \in S$, $R_\theta R_0 = R_{\theta+0} = R_\theta$ and $R_0 R_\theta = R_{0+\theta} = R_\theta$. Thus, R_0 (the 2×2 identity matrix) is the identity element.

4. **Inverse Element:** For any $R_\theta \in S$, we seek an $(R_\theta)^{-1} = R_\phi \in S$ such that $R_\theta R_\phi = R_0$. From the closure property, $R_\theta R_\phi = R_{\theta+\phi}$. So we need $R_{\theta+\phi} = R_0$. This means the matrix $R_{\theta+\phi}$ must be I_2 . This occurs when $\theta + \phi = 2k\pi$ for some integer k . We can choose $k = 0$, which gives $\phi = -\theta$. The matrix $R_{-\theta} = \begin{pmatrix} \cos(-\theta) & -\sin(-\theta) \\ \sin(-\theta) & \cos(-\theta) \end{pmatrix} = \begin{pmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{pmatrix}$. Since $\theta \in \mathbb{R}$, it follows that $-\theta \in \mathbb{R}$, so $R_{-\theta} \in S$. We verify: $R_\theta R_{-\theta} = R_{\theta+(-\theta)} = R_0$. Similarly, $R_{-\theta} R_\theta = R_{(-\theta)+\theta} = R_0$. Each element in S has an inverse in S . The inverse axiom holds.

Since all four group axioms are satisfied, $(S, \times)$ is a group. This group is known as $SO(2)$, the Special Orthogonal group of 2×2 real matrices.

Exercise 2: Rotational Symmetries in 2D

Let $\mathcal{G} = C_4$ be the group of 90° rotations in the two-dimensional plane, acting on square images of size $H \times W$ (assume $H = W$ for simplicity). Denote by g_{90} the element representing a clockwise rotation by 90° . The action of $g \in C_4$ on an image $\mathbf{I}$ is written $g \cdot \mathbf{I}$.

- (a) Suppose you have a filter (e.g. a Sobel filter) that detects horizontal edges. Let $f(\mathbf{I})$ be the feature map obtained by applying this filter to an image $\mathbf{I}$. If you apply the filter to a 90°-rotated image, $f(g_{90} \cdot \mathbf{I})$, how would you expect this result to relate to $g_{90} \cdot f(\mathbf{I})$ (i.e. the original feature map rotated by 90°)? Is this an example of equivariance?
- (b) A standard convolutional neural network (CNN) layer is typically composed of a convolution with learned filters, a non-linear activation (e.g. ReLU), and possibly a pooling operation. Is such a CNN layer (with filters learned from data without explicit symmetry constraints) generally equivariant to C_4 rotations? Explain why or why not.
- (c) You are designing a neural network to analyse cell-microscopy images in which cell orientation is arbitrary and diagnostically irrelevant. Why would C_4 (or continuous $SO(2)$) equivariance or invariance be a desirable property for your network? Which parts of the network would you aim to make equivariant, and which parts invariant?

Solution

- (a) Let f be a horizontal edge detector. $g_{90} \cdot \mathbf{I}$ is image $\mathbf{I}$ rotated 90° clockwise. Horizontal features in $\mathbf{I}$ become vertical in $g_{90} \cdot \mathbf{I}$. $f(g_{90} \cdot \mathbf{I})$ applies the horizontal edge detector to the rotated image, thus highlighting features that were originally *vertical* in $\mathbf{I}$. $f(\mathbf{I})$ detects horizontal edges in $\mathbf{I}$. $g_{90} \cdot f(\mathbf{I})$ rotates this feature map. It highlights (now rotated) locations that corresponded to *horizontal* edges in $\mathbf{I}$. For example, if $\mathbf{I}$ is a horizontal line, $f(g_{90} \cdot \mathbf{I})$ would yield a near-zero response (as $g_{90} \cdot \mathbf{I}$ is a vertical line), while $g_{90} \cdot f(\mathbf{I})$ would be a rotated map of the detected horizontal line. Thus, $f(g_{90} \cdot \mathbf{I}) \neq g_{90} \cdot f(\mathbf{I})$. This specific operation f is **not** C_4 -equivariant. For equivariance, $f(g \cdot \mathbf{x}) = g \cdot f(\mathbf{x})$ must hold.
- (b) A standard CNN layer $\phi(\mathbf{I}) = \text{Pool}(\text{ReLU}(\text{Conv}(\mathbf{I})))$ is generally **not** equivariant to C_4 rotations.
- Conv($\mathbf{I}$) uses learned filters W . These filters are typically asymmetric and optimized for specific patterns in the training data, not constrained to be C_4 -symmetric (i.e., $W \neq g \cdot W$ for $g \in C_4$). Thus, $\text{Conv}_W(g \cdot \mathbf{I}) \neq g \cdot \text{Conv}_W(\mathbf{I})$. This is the primary reason for the lack of layer equivariance.
- Since the initial convolution with unconstrained filters is not C_4 -equivariant, the entire layer generally isn't.
- (c) For cell-microscopy images where cell orientation is arbitrary and diagnostically irrelevant, C_4 or $SO(2)$ symmetry is highly desirable:

- **Benefits:** Leads to better data efficiency (network doesn't relearn features at different orientations), improved generalization from smaller datasets, and inherent robustness to orientational variations.

Network Design:

- **Feature Extraction Layers (early/middle):** These should be **equivariant**. If the input $\mathbf{I}$ rotates ($g \cdot \mathbf{I}$), the feature maps $F(\mathbf{I})$ should transform correspondingly ($F(g \cdot \mathbf{I}) = g' \cdot F(\mathbf{I})$, where g' might involve channel permutations for some group convolutions). This ensures features are detected consistently regardless of orientation.
- **Classification/Prediction Layers (final):** These should produce an **invariant** output. The final diagnosis $O(\mathbf{I})$ should be the same if the input is rotated ($O(g \cdot \mathbf{I}) = O(\mathbf{I})$).

C_4 vs. $SO(2)$: $SO(2)$ (continuous rotation) is theoretically ideal for arbitrary orientations but more complex (e.g., steerable CNNs). C_4 (discrete 90° rotations) is simpler and can be a good approximation.

Exercise 3: Equivariance and Invariance

- (a) Consider the task of object detection in images. An image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ is input, and the output is a set of bounding boxes. If the image is translated, how should the bounding boxes ideally transform? Does this describe equivariance or invariance with respect to the translation group? Explain.
- (b) Consider the task of image classification. An image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ is input, and the output is a class label (e.g., "cat" or "dog"). If the image is slightly rotated, should the class label change? Does this ideally describe equivariance or invariance with respect to a rotation group?

Solution

- (a) Consider the task of object detection.
 - **Input:** An image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$.
 - **Output:** A set of bounding boxes $\mathcal{B} = \{B_1, B_2, \dots, B_k\}$. Each bounding box B_i can be defined by coordinates, e.g., $(x_{center}, y_{center}, width, height)$.
 - **Transformation Group:** The translation group in 2D. Let T_v denote a translation by a vector $v = (v_x, v_y)$.

If an image $\mathbf{x}$ is translated by v to $T_v \cdot \mathbf{x}$, any object present in the image is also translated by v . Ideally, the object detection model ϕ should detect these translated objects. The bounding boxes for these objects should correspondingly be translated. Specifically, if $B = (x_c, y_c, w, h)$ is a bounding box in the original image $\mathbf{x}$, then in the translated image $T_v \cdot \mathbf{x}$, the corresponding bounding box $T_v \cdot B$ should be $(x_c + v_x, y_c + v_y, w, h)$. So, if $\phi(\mathbf{x}) = \{B_1, \dots, B_k\}$ is the set of bounding boxes for $\mathbf{x}$, then for the translated image $T_v \cdot \mathbf{x}$, the output should ideally be $\phi(T_v \cdot \mathbf{x}) = \{T_v \cdot B_1, \dots, T_v \cdot B_k\}$. This can be written as $\phi(T_v \cdot \mathbf{x}) = T_v \cdot \phi(\mathbf{x})$, where T_v acts on a set of boxes by translating each box in the set.

This describes **equivariance**. The transformation applied to the input (translation of the image) results in an equivalent transformation (translation by the same vector) applied to the output (the set of bounding boxes). (This assumes objects are not translated out of image boundaries, or new objects translated in, which are edge cases;)

- (b) Consider the task of image classification.
 - **Input:** An image $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$.
 - **Output:** A class label L (e.g., "cat", "dog").
 - **Transformation Group:** A rotation group (e.g., $SO(2)$ or a discrete subgroup). Let R denote a slight rotation.

If an image $\mathbf{x}$ depicting a "cat" is slightly rotated to $R \cdot \mathbf{x}$, it is still an image of a "cat". The semantic content relevant for classification is typically not altered by small rotations. Therefore, the class label assigned by an ideal image classification model ϕ should not change. So, if $\phi(\mathbf{x}) = L$ is the class label for $\mathbf{x}$, then for the slightly rotated image $R \cdot \mathbf{x}$, the output should ideally be $\phi(R \cdot \mathbf{x}) = L$.

This describes **invariance**. The transformation applied to the input (slight rotation of the image) results in no change to the output (the class label). This is crucial for robust classification, ensuring that minor variations in object orientation do not lead to misclassification.

Exercise 4: Coding Exercise: Rotation Equivariance and Group Convolutions

Solve the exercises in the accompanying file "exercise4.ipynb".

Solution

See the accompanying file "exercise4_solution.ipynb".

Exercise 5: Applications

You are designing a CNN for detecting lung nodules in chest X-rays. The nodules can appear at any orientation.

- (a) Explain why rotation equivariance would be beneficial for this task.
- (b) Compare three approaches:
 - Standard CNN with data augmentation
 - Group-equivariant CNN with C_4 symmetry
 - Steerable CNN with $SO(2)$ symmetry

Discuss the trade-offs in terms of:

- Parameter efficiency
 - Computational cost
 - Expected performance
- (c) Which approach would you recommend and why? Consider that medical datasets are typically small.

Solution

- (a) Rotation equivariance is beneficial for detecting randomly oriented lung nodules because:
 1. **Enhanced Data Efficiency & Generalization:** The network inherently understands rotated features. Learning a nodule at one orientation means it implicitly recognizes it at other orientations covered by the equivariance group. This leads to better generalization from fewer samples.
 2. **Improved Robustness:** Feature representations remain consistent (up to a transformation) regardless of input orientation, leading to more reliable detection.
 3. **Learns Intrinsic Features:** The model focuses capacity on diagnostically relevant nodule characteristics, not on learning rotational variations.
- (b) Comparison of Approaches
 - **1. Standard CNN with Data Augmentation:**
 - **Parameter efficiency:** Low (must learn all orientations separately)
 - **Computational Cost:** High training (processing augmented data), standard inference

- **Performance:** Good with large datasets, but only approximates equivariance
 - **2. Group-equivariant CNN (C_4 symmetry):**
 - **Mechanism:** Layers are equivariant to 90° rotations (e.g., rotated filters, constrained weights).
 - **Parameter Efficiency:** Better than standard CNN. Filters effectively cover 4 orientations.
 - **Computational Cost:** Higher per-layer cost (training & inference) than standard CNNs, but may require fewer layers/filters overall.
 - **Performance:** True equivariance at $0^\circ, 90^\circ, 180^\circ, 270^\circ$. interpolation needed for intermediate angles.
 - **3. Steerable CNN ($SO(2)$ symmetry):**
 - **Mechanism:** Filters (e.g., based on circular harmonics) are designed for true equivariance to any continuous 2D rotation.
 - **Parameter Efficiency:** High (compact representation via basis functions)
 - **Computational Cost:** Highest (training & inference) due to complex filter operations.
 - **Performance:** Theoretically best for arbitrary orientations, especially with small datasets, due to true continuous equivariance.
- (c) Considering that medical datasets are typically small and nodules can appear at any orientation, the approach that best leverages prior knowledge about symmetry to maximize data efficiency is most desirable.

Steerable CNN with $SO(2)$ symmetry, provided the implementation complexity and computational resources are manageable.

1. **Best for Small Datasets and Arbitrary Orientations:** $SO(2)$ equivariance provides the strongest guarantee for handling arbitrary orientations. Since the network inherently understands continuous rotations, it can generalize from very few examples of a nodule to its appearance at any orientation.
2. **Maximizes Parameter Efficiency for Rotational Features:** It learns the most compact representation of features that vary by rotation.
3. **True Equivariance:** Unlike data augmentation which approximates invariance/equivariance, or C_4 which only handles discrete rotations, $SO(2)$ offers genuine equivariance for the continuous group of 2D rotations.

Exercise 6: Translation Equivariance

Let T_a denote translation by vector $\mathbf{a} \in \mathbb{R}^2$, and let $*$ denote the convolution operation. Assume zero-padding at the boundaries.

- (a) Prove that standard 2D convolution is translation-equivariant:

$$T_{\mathbf{a}}(f * k) = (T_{\mathbf{a}}f) * k$$

- (b) Given a 1D signal $x = [1, 2, 3, 4]$ and kernel $k = [1, -1]$, compute:

The convolution $y = x * k$

The translated signal $x' = T_1(x)$ (shift by 1)

Verify that $T_1(y) = x' * k$

Solution

- (a) To prove that standard 2D convolution is translation-equivariant, i.e., $T_{\mathbf{a}}(f * k) = (T_{\mathbf{a}}f) * k$, we use the discrete definition of convolution: $(f * k)[\mathbf{n}] = \sum_{\mathbf{m}} f[\mathbf{m}]k[\mathbf{n} - \mathbf{m}]$, and the translation operator: $(T_{\mathbf{a}}f)[\mathbf{n}] = f[\mathbf{n} - \mathbf{a}]$.

LHS:

$$\begin{aligned} (T_{\mathbf{a}}(f * k))[\mathbf{n}] &= (f * k)[\mathbf{n} - \mathbf{a}] \\ &= \sum_{\mathbf{m}} f[\mathbf{m}]k[(\mathbf{n} - \mathbf{a}) - \mathbf{m}] \\ &= \sum_{\mathbf{m}} f[\mathbf{m}]k[\mathbf{n} - \mathbf{a} - \mathbf{m}] \end{aligned}$$

RHS:

$$\begin{aligned} ((T_{\mathbf{a}}f) * k)[\mathbf{n}] &= \sum_{\mathbf{m}'} (T_{\mathbf{a}}f)[\mathbf{m}']k[\mathbf{n} - \mathbf{m}'] \\ &= \sum_{\mathbf{m}'} f[\mathbf{m}' - \mathbf{a}]k[\mathbf{n} - \mathbf{m}'] \end{aligned}$$

Let $\mathbf{m} = \mathbf{m}' - \mathbf{a}$, so $\mathbf{m}' = \mathbf{m} + \mathbf{a}$. Substituting into the RHS:

$$\begin{aligned} ((T_{\mathbf{a}}f) * k)[\mathbf{n}] &= \sum_{\mathbf{m}} f[\mathbf{m}]k[\mathbf{n} - (\mathbf{m} + \mathbf{a})] \\ &= \sum_{\mathbf{m}} f[\mathbf{m}]k[\mathbf{n} - \mathbf{m} - \mathbf{a}] \end{aligned}$$

Since LHS = RHS, convolution is translation-equivariant. Zero-padding handles boundary conditions.

- (b) Given signal $x = [1, 2, 3, 4]$ and kernel $k = [1, -1]$. Indices start at 0. Assume full convolution with zero-padding.

1. **Convolution** $y = x * k$: $y[0] = x[0]k[0] = 1 \cdot 1 = 1$
 $y[1] = x[0]k[1] + x[1]k[0] = 1(-1) + 2(1) = 1$
 $y[2] = x[1]k[1] + x[2]k[0] = 2(-1) + 3(1) = 1$
 $y[3] = x[2]k[1] + x[3]k[0] = 3(-1) + 4(1) = 1$
 $y[4] = x[3]k[1] + x[4]k[0] = 4(-1) + 0(1) = -4$
 Thus, $y = [1, 1, 1, 1, -4]$.
2. **Translated signal** $x' = T_1(x)$ (shift by 1, $x'[n] = x[n-1]$): $x' = [x[-1], x[0], x[1], x[2], x[3]] = [0, 1, 2, 3, 4]$.
3. **Convolution** $x' * k$: $(x' * k)[0] = x'[0]k[0] = 0 \cdot 1 = 0$
 $(x' * k)[1] = x'[0]k[1] + x'[1]k[0] = 0(-1) + 1(1) = 1$
 $(x' * k)[2] = x'[1]k[1] + x'[2]k[0] = 1(-1) + 2(1) = 1$
 $(x' * k)[3] = x'[2]k[1] + x'[3]k[0] = 2(-1) + 3(1) = 1$
 $(x' * k)[4] = x'[3]k[1] + x'[4]k[0] = 3(-1) + 4(1) = 1$
 $(x' * k)[5] = x'[4]k[1] + x'[5]k[0] = 4(-1) + 0(1) = -4$
 Thus, $x' * k = [0, 1, 1, 1, 1, -4]$.
4. **Translated output** $T_1(y)$ ($T_1(y)[n] = y[n-1]$): $T_1(y) = [y[-1], y[0], y[1], y[2], y[3], y[4]] = [0, 1, 1, 1, 1, -4]$.
5. **Verification:** $T_1(y) = [0, 1, 1, 1, 1, -4]$
 $x' * k = [0, 1, 1, 1, 1, -4]$
 They are identical, confirming $T_1(y) = x' * k$.